

Actividad – Implementación de Listas, Pilas y Colas

Presentada por: Gabriela Castiblanco Castiblanco

1. Introducción

Este documento presenta la evidencia de la actividad solicitada, en la cual se implementó un sistema de control de turnos para la Universidad Compensar utilizando estructuras de datos dinámicas como listas, pilas o colas. El programa fue desarrollado completamente en Java dentro del entorno NetBeans.

2. Descripción del Programa

El software permite: Generar automáticamente un nuevo turno y agregarlo a la cola. Asignar el turno al operario disponible cuya especialidad coincida con el tipo de asesoría solicitada. Eliminar un turno cuando ha sido atendido, liberando al operario correspondiente. Mostrar todos los turnos pendientes.

3. Estructuras de Datos Utilizadas

Se emplearon las siguientes estructuras de datos: **Cola (Queue)**: Para manejar los turnos pendientes de forma FIFO. **Listas dinámicas (ArrayList)**: Para almacenar y gestionar los operarios. **Clases personalizadas**: Para representar turnos y operarios.

4. Funcionamiento del Menú Principal

El sistema contiene un menú interactivo basado en *switch-case*, el cual permite realizar todas las operaciones del sistema: Generar turnos Asignar turnos Finalizar turnos Mostrar turnos Salir

5. Capturas de Evidencia

```
run:

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 1
Tipo de turno (asesoria / caja): asesoria
Turno generado: 1 (asesoria)

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 2
Turno asignado correctamente:
Turno 1 (asesoria) - Operario: Ana
```

```
turnoss (run) #4 ^ turnoss (run) #3 ^ turnoss
run:

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 1
Tipo de turno (asesoria / caja): asesoria
Turno generado: 1 (asesoria)

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 1
Tipo de turno (asesoria / caja): asesoria
Turno generado: 2 (asesoria)

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 2
Turno asignado correctamente:
Turno 1 (asesoria) - Operario: Ana

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 2
Turno asignado correctamente:
Turno 1 (asesoria) - Operario: Laura
```

```

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 4
Turno 1 (asesoria) - Operario: Laura
Turno 2 (asesoria) - Operario: null

```

```

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 3
Turno finalizado: 1

```

```

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 3
Turno finalizado: 2

```

```

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 3
Turno finalizado: 2

```

```

===== MENU PRINCIPAL =====
1. Generar nuevo turno
2. Asignar turno a operario
3. Finalizar turno
4. Mostrar turnos pendientes
5. Salir
Seleccione una opción: 4
No hay turnos en espera.

```

6. Código Implementado El código completo se incluye en el archivo fuente entregado con esta evidencia.
7. package turnoss;
- 8.
9. import java.util.*;
- 10.
11. public class Turnoss {
- 12.

```
13.
14. static class Turno {
15.     private int numero;
16.     private String tipo;
17.     private String operarioAsignado;
18.
19.     public Turno(int numero, String tipo) {
20.         this.numero = numero;
21.         this.tipo = tipo;
22.     }
23.
24.     public void asignarOperario(String nombre) {
25.         this.operarioAsignado = nombre;
26.     }
27.
28.     @Override
29.     public String toString() {
30.         return "Turno " + numero + " (" + tipo + ") - Operario: " + operarioAsignado;
31.     }
32. }
33.
34.
35. static class Operario {
36.     private String nombre;
37.     private String especialidad;
38.     private boolean disponible = true;
39.
40.     public Operario(String nombre, String especialidad) {
41.         this.nombre = nombre;
42.         this.especialidad = especialidad;
43.     }
44.
45.     public boolean estaDisponible() {
46.         return disponible;
47.     }
48.
49.     public void ocupar() {
50.         disponible = false;
51.     }
```

```
52.
53.     public void liberar() {
54.         disponible = true;
55.     }
56.
57.     public String getNombre() {
58.         return nombre;
59.     }
60.
61.     public String getEspecialidad() {
62.         return especialidad;
63.     }
64. }
65.
66.
67. static class SistemaTurnos {
68.     private Queue<Turno> colaTurnos = new LinkedList<>();
69.     private List<Operario> operarios = new ArrayList<>();
70.     private int contadorTurnos = 1;
71.
72.     public void agregarOperario(Operario o) {
73.         operarios.add(o);
74.     }
75.
76.     public void generarTurno(String tipo) {
77.         Turno nuevo = new Turno(contadorTurnos++, tipo);
78.         colaTurnos.add(nuevo);
79.         System.out.println("Turno generado: " + nuevo.numero + " (" + tipo + ")");
80.     }
81.
82.     public void asignarTurno() {
83.         if (colaTurnos.isEmpty()) {
84.             System.out.println("No hay turnos en la cola.");
85.             return;
86.         }
87.
88.         Turno turno = colaTurnos.peek();
89.
90.
```

```
91.         for (Operario o : operarios) {
92.             if (o.estaDisponible() &&
103.                 o.getEspecialidad().equalsIgnoreCase(turno.tipo)) {
93.
94.                 turno.asignarOperario(o.getNombre());
95.                 o.ocupar();
96.
97.                 System.out.println("Turno asignado correctamente:");
98.                 System.out.println(turno);
99.
100.                return;
101.            }
102.        }
103.
104.        System.out.println("No hay operarios disponibles para este tipo de
        turno.");
105.    }
106.
107.    public void finalizarTurno() {
108.        if (colaTurnos.isEmpty()) {
109.            System.out.println("No hay turnos para finalizar.");
110.            return;
111.        }
112.
113.        Turno atendido = colaTurnos.poll();
114.
115.        // Liberar operario asignado
116.        for (Operario o : operarios) {
117.            if (o.getNombre().equals(atendido.operarioAsignado)) {
118.                o.liberar();
119.                break;
120.            }
121.        }
122.
123.        System.out.println("Turno finalizado: " + atendido.numero);
124.    }
125.
126.    public void mostrarTurnos() {
127.        if (colaTurnos.isEmpty()) {
```

```
128.         System.out.println("No hay turnos en espera.");
129.         return;
130.     }
131.     for (Turno t : colaTurnos) {
132.         System.out.println(t);
133.     }
134. }
135. }
136.
137.
138. public static void main(String[] args) {
139.     Scanner sc = new Scanner(System.in);
140.     SistemaTurnos sistema = new SistemaTurnos();
141.
142.     // Crear operarios por defecto
143.     sistema.agregarOperario(new Operario("Ana", "asesoria"));
144.     sistema.agregarOperario(new Operario("Carlos", "caja"));
145.     sistema.agregarOperario(new Operario("Laura", "asesoria"));
146.
147.     int opcion;
148.
149.     do {
150.         System.out.println("\n===== MENU PRINCIPAL =====");
151.         System.out.println("1. Generar nuevo turno");
152.         System.out.println("2. Asignar turno a operario");
153.         System.out.println("3. Finalizar turno");
154.         System.out.println("4. Mostrar turnos pendientes");
155.         System.out.println("5. Salir");
156.         System.out.print("Seleccione una opción: ");
157.
158.         opcion = sc.nextInt();
159.         sc.nextLine();
160.
161.         switch (opcion) {
162.             case 1:
163.                 System.out.print("Tipo de turno (asesoria / caja): ");
164.                 String tipo = sc.nextLine();
165.                 sistema.generarTurno(tipo);
166.                 break;
```

```
167.
168.         case 2:
169.             sistema.asignarTurno();
170.             break;
171.
172.         case 3:
173.             sistema.finalizarTurno();
174.             break;
175.
176.         case 4:
177.             sistema.mostrarTurnos();
178.             break;
179.
180.         case 5:
181.             System.out.println("Saliendo del sistema...");
182.             break;
183.
184.         default:
185.             System.out.println("Opción no válida.");
186.     }
187.
188. } while (opcion != 5);
189.
190. }
191. }
192.
```

7.Conclusión

La actividad permitió aplicar conceptos fundamentales de estructuras de datos como colas y listas dinámicas dentro de un caso real. El sistema de turnos desarrollado cumple con los requerimientos y demuestra el uso adecuado del manejo de memoria dinámica y la lógica de asignación de recursos.